

STAT:5400 Section 2.4 — LLM APIs and Prompting

Boxiang Wang

2026-09-16

1 What a Language Model Is

1.1 The one-sentence version

- A **large language model** writes text one token at a time, each token chosen using everything written before it.
- **Pretraining** trains the model from scratch on one task: predict the next token, over a very large amount of text.
- **Reasoning models** add reinforcement learning after pretraining, rewarded for correct answers rather than for matching text. They think in hidden tokens before replying. You pay for these tokens, but you do not see them.
- Modern models are built on the **Transformer** architecture, introduced in 2017.
- Most current models are also **multimodal**: they read images, audio, and video as well as text, and some generate them.

Source: Vaswani et al., *Attention Is All You Need*, 2017. <https://arxiv.org/abs/1706.03762>

1.2 History and evolution

- **1966 to 1996, rules and early neural networks.** [ELIZA](#) (1966) was a chatbot built from hand-written rules. [Recurrent neural networks](#) (1980s and 1990s) read text one word at a time.
- **1997 to 2012, better memory.** [LSTM](#) (1997) let a network keep information over longer stretches of text.
- **2013 to 2017, embeddings and the Transformer.** [Word2Vec](#) (2013) turned words into vectors. The [Transformer](#) (2017) became the core architecture of nearly every modern model.

- **2018 to 2021, pretraining at scale.** [BERT](#) (Google, 2018); [GPT-1](#) to GPT-3 (OpenAI, 2018 to 2020). [GPT-3](#) showed that a large enough model can learn a task from a few examples in the prompt.
- **2022 to 2023, chatbots.** [ChatGPT](#) (November 2022) brought prompt-based use to everyone. [GPT-4](#) and Anthropic's [Claude](#) followed in March 2023, Google's [Gemini](#) in December. Meta's [LLaMA](#) (February 2023) and [Mistral 7B](#) (September 2023) made strong models free to download, shared through [Hugging Face](#).
- **2024 onward, multimodal.** Gemini was built to read text, images, audio, and video from the start. [GPT-4o](#) (2024) added real-time voice; [Claude](#) reads images and PDFs. Image and video generators such as [Stable Diffusion](#) (2022) and [Sora](#) (2024) use related ideas.
- **2024 onward, reasoning.** [OpenAI o1](#) (September 2024) was trained to think before answering. [DeepSeek-R1](#) (January 2025) came close with open weights and a much lower reported training cost.
- **2025 onward, agents and open weights.** Models now run tools and code on their own: [Claude Code](#) and [OpenAI Codex](#) (2025), Section 7. Many of the strongest open-weight models now come from Chinese labs: [Qwen](#) (Alibaba), [DeepSeek](#), and [Kimi](#) (Moonshot AI).

Source: adapted from GeeksforGeeks, *History and Evolution of LLMs*, <https://www.geeksforgeeks.org/blogs/history-and-evolution-of-llms/>, with entries after 2022 added from the providers' announcements.

1.3 Two ways a model reaches you

- **Closed weights.** You call an API and never see the weights. [OpenAI GPT](#), [Anthropic Claude](#), [Google Gemini](#), [xAI Grok](#).
- **Open weights.** You can download, run, and fine-tune the model yourself, subject to its licence. [Meta Llama](#), [Google Gemma](#), [Mistral](#), [Alibaba Qwen](#), [DeepSeek](#), [Moonshot Kimi](#), [OpenAI gpt-oss](#).
- Some companies do both: Google has Gemini and Gemma; OpenAI has GPT and [gpt-oss](#).
- The largest open-weight models need a cluster of GPUs. Their small versions run on a laptop, or on IDAS.

2 Keys, Tokens, and Cost

2.1 API keys

Section 1.1: Set up your accounts.

- Read the prices: <https://platform.openai.com/docs/pricing>.

- Set a hard budget limit on the account. Do this before your first API call.

2.2 What a token costs

- Billing is per **token**, counted on both what you send and what comes back.
- One token is roughly 0.75 words. One million tokens is about 3000 double-spaced pages.

```
prompt <- "Explain what a p-value means in one sentence for a beginner."

words <- length(strsplit(prompt, "\\s+")[1])
tokens <- round(words / 0.75)
c(words = words, approx_tokens = tokens)
```

 Edit — put in a price per million tokens from the pricing page and a number of calls you might make in a homework, and see what the assignment would cost.

```
price_per_million <- 0.15      # dollars, input tokens; check the current page
calls <- 500

round(tokens * calls / 1e6 * price_per_million, 4)
```

For example, *The Elements of Statistical Learning* has 745 pages.

```
book_tokens <- 745 * 500 / 0.75  # pages, words per page, words per token
book_tokens
```

That is about half a million tokens. The block below gives the cost of sending the whole book once, with the cheapest and with the most expensive model.

```
# dollars per million input tokens, September 2026; these change often
per_million <- c(nano = 0.05, pro = 30.00)

round(book_tokens / 1e6 * per_million, 2)
```

- For the same text, the cheapest model costs about two cents and the most expensive about fifteen dollars.
- Output costs more than input, usually eight to ten times more, so a model that answers at length is the expensive one.
- A conversation resends its whole history every turn, so you pay for the book again on the next question.

The rule from Section 1.1 still holds. A key is a password. It lives in the environment, never in a file you can see. Read it with `Sys.getenv("OPENAI_API_KEY")` in R or `os.getenv` in Python, and never print it.

2.3 Setting up IDAS

STAT:5400 students can run every code block on this page on [IDAS](#). Do step 1 once. Do step 2 each time you start a new IDAS session.

1. **Save your OpenAI key.** Do this once, either in the R console or in a terminal on IDAS. The first line adds a fake key to the file `~/.Renviron`. The second opens the file: replace the fake key with your own and save. R reads this file when it starts, and the IDAS Python file reads it too.

In the R console:

```
cat("OPENAI_API_KEY=sk-fake-replace-with-your-key\n", file = "~/.Renviron", append = TRUE)
file.edit("~/.Renviron")           # replace the fake key with your own, then save
Sys.chmod("~/.Renviron", "600")    # only you can read the file
```

Or in a terminal:

```
echo 'OPENAI_API_KEY=sk-fake-replace-with-your-key' >> ~/.Renviron
nano ~/.Renviron          # replace the fake key with your own; Ctrl+O saves, Ctrl+X exits
chmod 600 ~/.Renviron    # only you can read the file
```

Then restart R (Session > Restart R). After the restart, `nzchar(Sys.getenv("OPENAI_API_KEY"))` returns TRUE if the key is set.

In a Jupyter notebook, nothing reads that file for you, and the notebook does not find the class copy of `openai` by itself. Start the notebook with this cell:

```
import sys, os, getpass
sys.path.insert(0, os.path.expanduser("~/classdata/models/python")) # class packages
from openai import OpenAI

os.environ["OPENAI_API_KEY"] = getpass.getpass("OpenAI key: ")
client = OpenAI()
```

The key then lives in memory only, and is never saved inside the notebook. The file `s2p4_idas.py` needs none of this: it reads `~/.Renviron` and finds the class packages itself.

2. **Start the class models.** Open a terminal on IDAS and type the line below. It starts Ollama with the open-weight models installed for the class, then lists them. Type `source`, not `bash`, or the `ollama` command is not found afterwards.

```
source ~/classFiles/notes/section2/run/s2p4_idas_models.sh
```

3. **Run the code.** All the code on this page is in two files in your `classFiles` folder. Open `notes/section2/run/s2p4_idas.R` in RStudio and run it one section at a time. For Python, type `python ~/classFiles/notes/section2/run/s2p4_idas.py` in the terminal.

3 Your First Call

3.1 Hello world

Run in Codespace — these need a key and a network, so they run in the Codespace.

3.1.1 R

```
library(ellmer)

chat <- chat_openai(model = "gpt-5-nano")
chat$chat("Write a one-sentence bedtime story about a unicorn.")
```

3.1.2 Python

```
from openai import OpenAI

client = OpenAI() # reads OPENAI_API_KEY from the environment
response = client.responses.create(
    model="gpt-5-nano",
    input=[{"role": "user", "content":
        "Write a one-sentence bedtime story about a unicorn."}],
)
print(response.output_text)
```

Run it twice. The two answers differ, because the model samples. We come back to why in Section 3.1.

3.2 Open-weight models

Most providers accept the OpenAI format, so only the address and the model name change. That includes open-weight models that you run yourself, through [Ollama](#). They need no key and cost nothing.

- On IDAS, start the class models first, as in [Setting up IDAS](#).
- In the Codespace, llama3.2:1b is already installed.

3.2.1 R

```
small <- chat_ollama(model = "llama3.2:1b")      # 1 billion parameters
small$chat("Give me 3 distributions in the exponential family.")

bigger <- chat_ollama(model = "qwen3.5:4b",      # 4 billion parameters, IDAS only
  api_args = list(reasoning_effort = "none"))    # skip the thinking step
bigger$chat("Give me 3 distributions in the exponential family.")
```

3.2.2 Python

```
local = OpenAI(base_url="http://localhost:11434/v1", api_key="ollama")

resp = local.chat.completions.create(
    model="llama3.2:1b",                        # 1 billion parameters
    messages=[{"role": "user",
                "content": "Give me 3 distributions in the exponential family."}])
print(resp.choices[0].message.content)

resp = local.chat.completions.create(
    model="qwen3.5:4b", reasoning_effort="none", # IDAS only; skip the thinking step
    messages=[{"role": "user",
                "content": "Give me 3 distributions in the exponential family."}])
print(resp.choices[0].message.content)
```

- `base_url` says where to send the request. In R, `chat_ollama()` already knows the address.
- `api_key="ollama"` is a placeholder. The local server ignores it, but the library needs something there.
- `reasoning_effort = "none"` turns off the thinking step. On a CPU, thinking can take several minutes.

Compare the answers with the hosted model in [Hello world](#). On IDAS, the 1B model answers in a few seconds and is often wrong. The 4B model takes up to a minute and usually names the right distributions, but it can still write a formula wrong. A hosted model is faster and better, and this difference is what you pay for.

A small local model is also useful for testing your code. It costs nothing and needs no key, so get your script working with it first, then switch to a hosted model.

4 Prompting

4.1 Rules of thumb

Prompt engineering means writing instructions clear enough that the model does what you need, reliably. The output is random, so a prompt that works once may not always work. The seven rules below follow the current guides from OpenAI and Anthropic. Open each one for a bad and a better example.

1. Be clear and specific. Say who the answer is for, how long it should be, and what form it should take.

i Bad and better

If a colleague with no background would be confused by your prompt, the model will be too.

Bad:

Explain the bootstrap. Keep it fairly short but somewhat detailed.

Better:

Explain the nonparametric bootstrap to a first-year statistics graduate student, in at most 150 words, as three bullet points, and end with one line of R code that resamples a vector.

Words like short, detailed, or clear cannot be checked. A number of words, a number of bullet points, and a named reader can.

2. Give context, and say why an instruction matters.

i Bad and better

With the reason, the model can also handle cases your instruction did not mention.

Bad:

Do not use the word "significant".

Better:

The readers are doctors, who read "significant" as "clinically important".
When you mean the result of a test, write "statistically significant at the 5% level" instead.

3. Separate the parts of a long prompt, and put the question last.

i Bad and better

Use Markdown headings or tags to mark the instructions, the examples, and the data. With a long document, put the document first and the question after it.

Bad:

Summarize the main finding and list any statistical problems.

Here is the paper: <forty pages pasted here, with nothing marking where they end>

Better:

<paper>

... forty pages ...

</paper>

Summarize the main finding of the paper above in two sentences. Then list any statistical problems you see, each with the page where it appears.

Anthropic reports that putting the question after a long document can improve answers by up to 30 percent. The few-shot classifier below uses the same structure: headings for the parts, and tags around each example.

4. Say what to do, including what to do when the model does not know.

i Bad and better

A list of things to avoid leaves the model guessing what you want.

Bad:

You are a tutor. Do not give the answer. Do not make things up.

Better:

You are a tutor. Explain the idea behind the answer in at most 100 words, then ask the student what they would try next. If the question needs information you do not have, say so and ask for it.

Allowing the model to say it does not know is one of the simplest ways to reduce made-up answers.

5. Start without examples. Add a few if needed, and fine-tune last.

i Bad and better

Try the cheap step first, and move on only when it fails.

Bad:

Collect 500 labeled claims and fine-tune a model, before trying a prompt.

Better:

Zero-shot: Label this claim as Supported, Unsupported, or Unclear: "..."

Few-shot: the same instruction, plus three to five varied, labeled examples
(as in the few-shot classifier below)

Fine-tune: only if few-shot still fails on many claims

Reasoning models often need no examples at all. Fine-tuning is done outside the chat: you send the provider hundreds of labeled examples, and it trains a new version of the model for you. It costs time and money, so it comes last.

6. Let a reasoning model do its own thinking.

i Bad and better

A reasoning model already thinks before it answers. Writing out the steps for it can make the answer worse, not better.

Bad:

Think step by step. First write down the density, then compute $E(X)$, then $E(X^2)$, then check every step twice. What is the variance of a Uniform(0, 1) random variable?

Better:

What is the variance of a Uniform(0, 1) random variable? Give the answer and one sentence of explanation.

For a harder problem, raise the reasoning effort in the API call, as `/effort` does for coding agents, instead of adding steps to the prompt. Asking for steps can still help older models and small open-weight models.

7. Test the prompt on many examples, not one.

i Bad and better

The output is random, so one good answer proves little. Test the prompt on many examples, and count how often it is right.

Bad:

Try the prompt on one claim. The label looks right, so use it.

Better:

Write 20 claims whose labels you are sure of. Run prompt A and prompt B on all 20, three times each, and compare the proportion of correct labels.

The proportion correct is an estimate with its own standard error. With 20 claims, a difference of one or two correct labels may be noise.

Two more habits help.

- **Choose a recent model.** A newer small model is often better, and cheaper, than an older one: `gpt-5-nano` (2025) costs \$0.05 per million input tokens, and `gpt-3.5-turbo` (2023) costs \$0.50.
- **Ask the model to draft the prompt, then edit it.** Describe the task to a chatbot and ask it to write the prompt. Then read the result sentence by sentence, and fix anything that is not what you mean.

i How to ask a model to write a prompt

Describe the task, say what the answer must look like, and let the model ask you questions first.

To draft a new prompt, send something like this to ChatGPT or Claude:

To improve a prompt you already have:

Here is my prompt:

<paste your prompt here>

It sometimes answers with a full sentence instead of one word. Tell me which phrases to add or delete to fix this, then give the revised prompt.

The providers also have tools for this: OpenAI's [prompt optimizer](https://developers.openai.com/api/docs/guides/prompt-engineering) in its Playground (log in first), and Anthropic's [metaprompt notebook](https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices), which writes a first draft of a prompt for Claude.

Source: OpenAI, *Prompt engineering* <https://developers.openai.com/api/docs/guides/prompt-engineering> and *Reasoning best practices* <https://developers.openai.com/api/docs/guides/reasoning-best-practices>; Anthropic, *Prompting best practices* <https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices>; checked September 2026.

4.2 Roles

Each message has a role, and some roles have higher priority than others.

- **platform**: the provider's own rules. You cannot override them, which is why a prompt telling the model to ignore its safety rules does not work.
- **developer**: your instructions. Sent as the `instructions` argument, or as a message with role `developer`.
- **user**: the end user's request, which ranks below your instructions.
- **assistant**: what the model said on earlier turns.

i system or developer?

They are the same level, under two names. `system` is the old name. Since the reasoning models arrived, the role is called `developer`, and current models expect that name. Older code using `system` still works, and most providers accept it, but write `developer` in anything new. In `ellmer` the argument is still called `system_prompt`, and it is sent at this level.

Source: OpenAI model spec, https://model-spec.openai.com/2025-02-12.html#chain_of_command

💡 Predict — the same question, two system prompts. Which one is more useful to a student who has to take an exam?

4.2.1 R

```
chat_a <- chat_openai(model = "gpt-5-nano",
  system_prompt = "You are a concise tutor.")
chat_a$chat("X follows a Uniform(0, 1). What is its variance?")

chat_b <- chat_openai(model = "gpt-5-nano",
  system_prompt = paste("You are a responsible tutor. Help your student",
    "but don't tell them the exact answers. "))
chat_b$chat("X follows a Uniform(0, 1). What is its variance?")
```

4.2.2 Python

```
for system in ["you are a concise tutor.",
  "you are a responsible tutor. Help your student "
  "but don't tell them the exact answers."]:
  response = client.responses.create(
    model="gpt-5-nano",
    input=[{"role": "developer", "content": system},
      {"role": "user", "content":
```

```

        "X follows a uniform (0, 1). What is its variance?"]])
print(response.output_text, "\n---")

```

4.3 The model does not remember earlier calls

- By default, every request is **stateless**: it starts from nothing, and the model sees only what you send in that request. It does not remember your earlier calls.
- Ask about “the university I just mentioned” in a second, separate call, and it has no idea.

4.3.1 R

Two `chat` objects are two separate conversations. One `chat` object is a memory.

```

library(ellmer)

first <- chat_openai(model = "gpt-5-nano")    # reads OPENAI_API_KEY
first$chat("Write a paragraph about the Department of Statistics and
           Actuarial Science at the University of Iowa.")

second <- chat_openai(model = "gpt-5-nano")   # a brand new conversation
second$chat("What is the name of the university I just mentioned?")

same <- chat_openai(model = "gpt-5-nano")
same$chat("Write two sentences about the University of Iowa.")
same$chat("What is the name of the university I just mentioned?")

# The same test with Claude. Only the function changes, and it needs ANTHROPIC_API_KEY.
# same <- chat_anthropic()
# same$chat("Write two sentences about the University of Iowa.")
# same$chat("What is the name of the university I just mentioned?")

```

4.3.2 Python

```

r1 = client.responses.create(
    model="gpt-5-nano",
    input=[{"role": "user", "content":
            "Write a paragraph about the Department of Statistics and Actuarial "
            "Science at the University of Iowa."}])
print(r1.output_text)

```

```
r2 = client.responses.create(
  model="gpt-5-nano",
  input=[{"role": "user", "content":
    "What's the name of the university that I just mentioned?"})
print(r2.output_text)      # it does not know
```

- A conversation is something *you* build, by sending the history back every time.
- `ellmer` does this for you: a `chat` object keeps the turns and resends them.
- Every LLM API works like this, not only OpenAI. The last lines of the R code, commented out, run the same test with Claude.

4.4 Conversation state

The model keeps nothing between requests, as the block above shows. To continue a conversation, the model has to see the earlier messages again. There are three ways to do this.

1. Pass the previous messages yourself. Write the earlier turns into the request, each with its role. The model answers as if the conversation had really happened.

4.4.1 R

```
chat <- chat_openai(model = "gpt-5-nano")
chat$set_turns(list(
  UserTurn("knock knock."),
  AssistantTurn("Who's there?")))
chat$chat("Felix")
```

4.4.2 Python

```
response = client.responses.create(
  model="gpt-5-nano",
  input=[{"role": "user",      "content": "knock knock."},
         {"role": "assistant", "content": "Who's there?"},
         {"role": "user",      "content": "Felix"}])
print(response.output_text)
```

2. Keep a history and resend it. Keep every message in a list. After each reply, add the reply and your next question to the list, then send the whole list again.

4.4.3 R

A `chat` object does this for you. It keeps the history on your computer and resends all of it with each new message.

```
chat <- chat_openai(model = "gpt-5-nano")
chat$chat("Tell me a joke about statistics.")
chat$chat("Tell me another.")
chat$get_turns()      # every turn so far: user, assistant, user, assistant
```

4.4.4 Python

```
history = [{"role": "user", "content": "Tell me a joke about statistics."}]
response = client.responses.create(model="gpt-5-nano", input=history)
print(response.output_text)

history.append({"role": "assistant", "content": response.output_text}) # add to the end
history.append({"role": "user", "content": "Tell me another."})
response = client.responses.create(model="gpt-5-nano", input=history)
print(response.output_text)
```

3. Let the provider keep it. OpenAI can store the conversation on its servers. You send only the new message and the ID of the last reply. You still pay for the earlier messages, because the model reads them again.

```
response = client.responses.create(
    model="gpt-5-nano",
    input="Tell me a joke about statistics.")
print(response.output_text)

second = client.responses.create(
    model="gpt-5-nano",
    previous_response_id=response.id,      # the ID of the last reply
    input="Explain why this is funny.")
print(second.output_text)
```

This is an OpenAI feature. `ellmer` has no counterpart, because its `chat` object always resends the history (way 2).

i The context window

Everything you resend is paid for again, and there is a ceiling.

The **context window** is the maximum number of tokens in one request, counting what you send, what comes back, and for some models the hidden reasoning. A long conversation eventually hits it, and the output gets truncated. Model limits are listed at <https://platform.openai.com/docs/models>.

4.5 Few-shot learning

- **Zero-shot:** you describe the task. **Few-shot:** you also show a handful of worked examples.
- The model reads the pattern from the examples and applies it. The model is not retrained. The examples are used for this request only.
- Diverse examples generalize better than similar ones.

Source: Brown et al. (2020), *Language Models are Few-Shot Learners*. <https://arxiv.org/pdf/2005.14165>

A classifier for statistical claims, built entirely out of three examples:

```
INSTRUCTIONS = """"# Identity

You are a helpful assistant that labels short statistical claims as
Supported, Unsupported, or Unclear based on the evidence provided.

# Instructions

* Only output a single word, with no additional formatting or commentary.
* Your response must be one of "Supported", "Unsupported", or "Unclear".

# Examples

<stat_claim id="example-1">
The new drug lowered systolic blood pressure
(mean difference -5.2 mmHg, 95% CI -8.1 to -2.3, p=0.001).
</stat_claim>
<assistant_response id="example-1">Supported</assistant_response>

<stat_claim id="example-2">
This study proves the two groups are different
(mean difference 1.1, 95% CI -0.6 to 2.8, p=0.20).
```

```

</stat_claim>
<assistant_response id="example-2">Unsupported</assistant_response>

<stat_claim id="example-3">
The treatment improved outcomes, but only a trend was observed (p=0.07).
</stat_claim>
<assistant_response id="example-3">Unclear</assistant_response>
"""

ALLOWED = {"Supported", "Unsupported", "Unclear"}

def label_claim(claim):
    resp = client.responses.create(model="gpt-5-nano",
                                   instructions=INSTRUCTIONS, input=claim)
    label = (resp.output_text or "").strip().split()[0].strip('"\'.,;:')
    if label not in ALLOWED:
        raise ValueError(f"Unexpected label {label!r}")
    return label

for c in ["We can conclude there is no effect because p=0.40.",
          "The results show a significant increase (95% CI 0.1 to 0.4).",
          "The intervention might help, but the CI crosses zero (95% CI -0.2 to 0.3)."]:
    print(label_claim(c))

```

The function checks the reply: the word must be one of the three allowed labels, or the function raises an error. The model may return a full sentence even when you asked for one word.

4.6 Sampling controls

At each step the model gives every token i in its vocabulary a score z_i , called a **logit**. It turns the scores into probabilities and draws the next token from them: one multinomial trial with V categories, where V is the size of the vocabulary.

Temperature $T > 0$ divides every logit before the softmax:

$$p_i(T) = \frac{\exp(z_i/T)}{\sum_{j=1}^V \exp(z_j/T)}, \quad \log \frac{p_i(T)}{p_j(T)} = \frac{z_i - z_j}{T}.$$

- Temperature divides every log odds by T .
- $T = 1$ gives the model's own distribution.

- As $T \rightarrow 0$, all the probability moves to the largest z_i , so the model always takes the most likely token. This is called greedy decoding.
- As $T \rightarrow \infty$, the distribution becomes uniform.

Top-p, or nucleus sampling, removes the long tail. Sort the probabilities, $p_{(1)} \geq p_{(2)} \geq \dots$, keep the smallest k with $p_{(1)} + \dots + p_{(k)} \geq p$, rescale those k probabilities to sum to 1, and draw from them.

An example. The text so far is

The p-value is 0.03, so at the 5% level we _____ the null hypothesis.

A real vocabulary has tens of thousands of tokens. Suppose only three have logits worth showing:

Next word	Logit z_i	$T = 0.5$	$T = 1$	$T = 2$
reject	2	0.867	0.665	0.506
accept	1	0.117	0.245	0.307
retain	0	0.016	0.090	0.186

- Only “reject” is correct. At $T = 2$, the model gives a wrong conclusion about half the time.
- With $T = 1$ and top-p = 0.9, the model keeps “reject” and “accept”, because $0.665 + 0.245 = 0.910 \geq 0.9$. After rescaling they have probabilities 0.731 and 0.269, and “retain” can no longer be drawn.
- Top-p did not remove the wrong word “accept”. It only cuts the tail. When a task has one right answer, use a low temperature.

Section 3.1 simulates draws like this one.

- Rules of thumb: temperature 0.2 with top-p 0.95 for coherent output; 0.9 and 0.99 for creative output; 0 when the task has one right answer.
- Very low and very high temperatures both cause problems. Too low, and the model may repeat the same text; too high, and the text may stop making sense.

i Some models do not accept these settings

Check your model before you set these. Requests to the GPT-5 family, including `gpt-5-nano`, raise an error if `temperature` or `top_p` is included. To set them you need a model that supports them, with reasoning effort turned off.

5 Structured Output

5.1 Ask for a type, not a string

- A model always replies with text. To pass its answer to `lm` or to a plot, you need numbers and columns, not a paragraph.
- Both `ellmer` and the OpenAI library let you describe the shape you want, called a **schema**, and they return data in that shape.
- With a schema, you no longer need the checking code from the few-shot example above.

5.1.1 R

```
review <- type_object(  
  label      = type_enum("One of Supported, Unsupported, Unclear.",  
                        c("Supported", "Unsupported", "Unclear")),  
  reason     = type_string("One short sentence."),  
  p_value    = type_number("The p-value mentioned, or -1 if none."))  
  
chat <- chat_openai(model = "gpt-5-nano")  
chat$chat_structured(  
  "This study proves the two groups differ (p = 0.20).",  
  type = review)
```

5.1.2 Python

```
from pydantic import BaseModel  
from typing import Literal  
  
class Review(BaseModel):  
    label: Literal["Supported", "Unsupported", "Unclear"]  
    reason: str  
    p_value: float  
  
resp = client.responses.parse(  
    model="gpt-5-nano",  
    input="This study proves the two groups differ (p = 0.20).",  
    text_format=Review)  
print(resp.output_parsed.label, resp.output_parsed.p_value)
```

6 Tool Calling

6.1 Giving the model a function to call

- A model is not a calculator. It can get simple arithmetic wrong.
- A **tool** is one of your functions, described well enough that the model can decide when to call it and with what arguments.
- You run the function. The model never touches your data or your session.

The loop has four steps, and you will meet it again in Section 7.

1. You send the question and the list of tools.
2. The model replies with a request: this tool, these arguments.
3. Your code runs the function and sends the result back.
4. The model uses the result to answer, or asks for another tool.

6.2 Watch a small model fail, then fix it

In 2024, many chatbots answered that 9.11 is larger than 9.9. Small models still make this mistake.

We ask the model the same question twice: first without a tool, and then with a tool `larger(a, b)` that compares the two numbers in R or Python.

Run in Codespace or IDAS — needs a local model. Run the code a few times. The answers change from run to run, and `llama3.2:1b` is wrong about half the time.

6.2.1 R

```
library(ellmer)

# no tool: the model answers from text alone
bare <- chat_ollama(model = "llama3.2:1b")
bare$chat("Which is larger, 9.9 or 9.11? Answer with just the number.")

# one tool: now the comparison happens in R
larger <- function(a, b) {
  a <- as.numeric(a); b <- as.numeric(b)           # small models sometimes send "9.9" as text
  if (a > b) a else if (b > a) b else "equal"
}

armed <- chat_ollama(model = "llama3.2:1b",
```

```

system_prompt = "Use the tool to compare numbers. Never compare them yourself.")
armed$register_tools(list(
  tool(larger, "Return the larger of two numbers.",
    arguments = list(a = type_number("First number."),
                      b = type_number("Second number."))))
armed$on_tool_request(function(req) cat("-> ", req$name, deparse1(req@arguments), "\n"))
armed$chat("Which is larger, 9.9 or 9.11?")

```

6.2.2 Python

```

import json
local = OpenAI(base_url="http://localhost:11434/v1", api_key="ollama")

def larger(a, b):
    a, b = float(a), float(b)          # small models sometimes send "9.9" as text
    return max(a, b)

TOOLS = [{"type": "function", "function": {
    "name": "larger",
    "description": "Return the larger of two numbers.",
    "parameters": {"type": "object",
        "properties": {"a": {"type": "number"}, "b": {"type": "number"}},
        "required": ["a", "b"]}}}]

msgs = [{"role": "system", "content": "Use the tool to compare numbers."},
        {"role": "user", "content": "Which is larger, 9.9 or 9.11?"}]

# steps 1 and 2: send the question with the tool, and read the model's request
reply = local.chat.completions.create(model="llama3.2:1b", messages=msgs, tools=TOOLS)
msgs.append(reply.choices[0].message)

# step 3: run the function and send its result back
for call in reply.choices[0].message.tool_calls or []:
    args = json.loads(call.function.arguments)
    print("->", call.function.name, args)
    msgs.append({"role": "tool", "tool_call_id": call.id,
        "content": str(larger(**args))})

# step 4: the model answers, using the result
final = local.chat.completions.create(model="llama3.2:1b", messages=msgs, tools=TOOLS)
print(final.choices[0].message.content)

```

What you should see:

- Without the tool, the model often answers 9.11, which is wrong.
- With the tool, the model asks to call `larger` with 9.9 and 9.11 (the line that starts with `->`), and then answers 9.9.
- In R, `ellmer` sends the result of the tool back to the model for you (steps 3 and 4 in Section 6.1). In Python you do this yourself: add the model's request and the result to the message list, and call the model again.

Why the tool helps

The model did not get better at comparing numbers.

With the tool, R or Python does the comparison, and they always get `9.9 > 9.11` right. The model only has to recognize that two numbers need to be compared, and pass them to the tool. In general, if a model often gets a calculation wrong, give it a tool that does the calculation.

If the small model does not use the tool

Small models such as `llama3.2:1b` often use tools incorrectly. The model may answer without calling `larger`, or pass the wrong numbers. This is a problem with the model, not with your code. The line that starts with `->` shows which numbers the model passed. In our tests on IDAS, `qwen3.5:4b` called the tool correctly every time, so use it when you need the tool to work.

6.3 A two-tool example

Now the question needs two tools: “What is the average weight of these cars?” We give the model two functions:

- `column_names()` returns the column names of the cars data.
- `column_mean(column)` returns the mean of one column.

The model cannot see the data. To answer, it first needs the column names, and then the mean of the right column.

Run in Codespace — needs a model.

6.3.1 R

```

library(ellmer)
Cars <- read.delim("notes/section2/data/Cars.dat")

column_names <- function() paste(names(Cars), collapse = ", ")
column_mean <- function(column) mean(Cars[[column]])

chat <- chat_openai(model = "gpt-5-nano",
  system_prompt = "Use the tools to compute. Never guess a number.")

chat$register_tools(list(
  tool(column_names,
    "List the column names of the cars data set. Call this first if unsure.",
    arguments = list()),
  tool(column_mean,
    "Return the mean of one numeric column of the cars data set.",
    arguments = list(column = type_string("Exact column name."))))))

chat$on_tool_request(function(req)
  cat("-> ", req$name, deparse1(req$arguments), "\n"))

chat$chat("What is the average weight of these cars?")

```

6.3.2 Python

```

import json, pandas as pd
Cars = pd.read_csv("notes/section2/data/Cars.dat", sep="\t")

TOOLS = [
  {"type": "function", "name": "column_names",
    "description": "List the column names of the cars data set.",
    "parameters": {"type": "object", "properties": {},
      "required": [], "additionalProperties": False},
    "strict": True},
  {"type": "function", "name": "column_mean",
    "description": "Return the mean of one numeric column of the cars data set.",
    "parameters": {"type": "object",
      "properties": {"column": {"type": "string"}},
      "required": ["column"], "additionalProperties": False},
    "strict": True},
]

```

```

def call_tool(name, args):
    if name == "column_names":
        return ", ".join(Cars.columns)
    if name == "column_mean":
        column = args["column"]
        if column not in Cars.columns:
            return f"There is no column named {column}." # the model reads this and retrieves
        return float(Cars[column].mean())

messages = [{"role": "user", "content": "What is the average weight of these cars?"}]
for step in range(5):
    resp = client.responses.create(model="gpt-5-nano", input=messages,
                                   instructions="Use the tools to compute. Never guess a number.",
                                   tools=TOOLS, parallel_tool_calls=False)

    messages += resp.output
    calls = [o for o in resp.output if o.type == "function_call"]
    if not calls:
        print(resp.output_text); break
    for c in calls:
        result = call_tool(c.name, json.loads(c.arguments or "{}"))
        print("->", c.name, c.arguments, "=>", result)
        messages.append({"type": "function_call_output",
                          "call_id": c.call_id, "output": json.dumps(result)})

```

What you should see: first a call to `column_names`, then a call to `column_mean` with `column = "Weight"`, and then an answer with the mean. The loop ends when the model answers without asking for a tool.

👉 By hand — the correct answer is the mean of the **Weight** column, which you can compute yourself in one line. Run the code and check that the model reports the same number. Done when you can say which tools the model called, in which order, and whether it ever guessed a number.

7 Working with a Coding Agent

7.1 Git

Command	What it tells you
<code>git status</code>	which files changed
<code>git diff</code>	exactly which lines changed

Command	What it tells you
<code>git add .</code> and <code>git commit -m "..."</code>	save a checkpoint
<code>git log --oneline</code>	the checkpoints so far
<code>git restore <file></code>	throw away everything since the last checkpoint

- Commit before you let an agent change anything. If the agent breaks something, you can go back to that commit.
- `git diff` is how you see what it did, line by line. `/review` reads the same thing.
- A `.gitignore` lists what never enters the repository: `.env`, large data, rendered output. Always include `.env`, so that your key never reaches GitHub.

7.2 Installing Codex and Claude Code

Both are command-line programs. One line installs either one and puts the command on your PATH, so you can type it from any folder.

```
# Claude Code
curl -fsSL https://claude.ai/install.sh | bash

# Codex
curl -fsSL https://chatgpt.com/codex/install.sh | sh
```

- If you already use Node, `npm install -g @anthropic-ai/claude-code` and `npm install -g @openai/codex` do the same job. The installers above carry their own runtime, so Node is not required.
- Check it worked with `claude --version` or `codex --version`. If the shell says command not found, open a new terminal. If it still fails, the install folder is not on your PATH, and you add it in `~/.zshrc` or `~/.bashrc`.
- The first run asks you to sign in. Claude Code takes a Claude subscription or `ANTHROPIC_API_KEY`. Codex takes a ChatGPT account or `OPENAI_API_KEY`.
- Both also install inside the Codespace terminal, so you can try them without touching your own machine.

7.3 Let the agent set it up

Set up git every time before you start an agent.

```
cd ~/projects/my-analysis # the folder you want to work in
claude                   # or: codex
```

🤖 Vibe — once it starts, ask for something like this.

Initialize a git repository here. Write a .gitignore for a project that uses both R and Python, excluding .env, .Rhistory, .Rproj.user, __pycache__, and rendered output. Then make the first commit.

Then check what the agent did:

```
git log --oneline      # exactly one commit?
git status             # working tree clean?
cat .gitignore         # is .env actually in there?
git ls-files | grep .env # nothing should come back
```

The last line is the most important. If a .env file was committed, the agent has just put a key into your history, and deleting the file later does not remove it. Done when you can say what it committed, what it left out, and how you know.

7.4 Chat, API, and coding agents

There are three ways to use a model.

- **Chat.** You type into a web page.
- **API.** You write the call inside a script that can be rerun, versioned, and read by someone else.
- **Coding agent.** The model runs the loop on your machine. It reads your files, writes new ones, runs commands, reads the errors, and tries again. [OpenAI Codex](#) and [Anthropic Claude Code](#) are the two common ones. [Copilot](#) in the Codespace is a lighter version of the same idea.
- The main difference is who is in control. With the API, your script decides every step. A coding agent decides the steps itself, and it changes your files directly.
- All three leave a record you can check later. A chat can be saved or shared as a web page. With the API, your script and its output are the record. A coding agent keeps a transcript of every step, and Section 7 is about reading one.
- In this course: coding agents are welcome on 🤖 Vibe blocks, not on 🖐 By hand blocks, and whatever they write, you run and check yourself.

You use both from a terminal. Most of the time you type ordinary English. To control the tool itself, for example to switch models or start over, you type a **slash command**. Type / alone to see the list for your version, because it changes often. Six are worth knowing on day one.

Command	What it does	Use it when
<code>/help</code>	lists every command	you forget the rest of this table
<code>/init</code>	writes a memory file, <code>CLAUDE.md</code> or <code>AGENTS.md</code>	you start work in a new repository
<code>/model</code>	switches models	a task needs more care, or more speed
<code>/clear</code>	throws the conversation away and starts fresh	the agent is going in circles
<code>/effort</code>	sets how long the model thinks before answering	the task is hard, or the bill is too high
<code>/review</code>	reviews the changes instead of making more	you are about to commit

`/effort` takes `low` through `max`. Low is fast and cheap, high thinks longer. It is a Claude Code command; in Codex the same dial sits inside `/model`. Pairing a small model on low effort for mechanical edits with a large one on high effort for design is an easy way to lower the cost.

A session looks like this.

```
> /init
wrote AGENTS.md

> read notes/section2/data/Cars.dat and write an R function
that returns the mean of any numeric column

... writes R/cars.R, runs it, hits one error, fixes it ...

> /review

> /clear
```

`/init` is the most important command for this course. The memory file is plain text, and it lives in the repository. The instructions the agent followed end up under version control, next to the code it wrote. This way, someone else can reproduce your AI-assisted analysis.

i Going further: carrying work across sessions

A real task often takes more than one session. Three commands and one file help you carry it over.

- `/compact` replaces a long conversation with a summary and keeps going. Use it

when the session becomes slow or the context is full.

- `/resume` reopens an earlier session. At the shell, `--continue` reopens the last one.
- `/export` writes the whole transcript to a text file.

Session history is saved only on the machine where you ran it, and it does not follow you to another computer. Keep what matters in the repository instead: the memory file holds your instructions, and an exported transcript records what the agent actually did.

This is the same reason we use `set.seed`: other people should be able to get the same result.